

Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 4 - Intermediate Code and Interpreter

Semester II tahun 2025/2026



Oleh:

Kelompok ReguExceptional:

Wafiq Hibban Robbany	13524016
Muhammad Nafis Habibi	13524018
An-Dafa Anza Avansyah	13524038
Wildan Abdurrahman Ghazali	13524054

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

04 Juni 2026

Daftar Isi

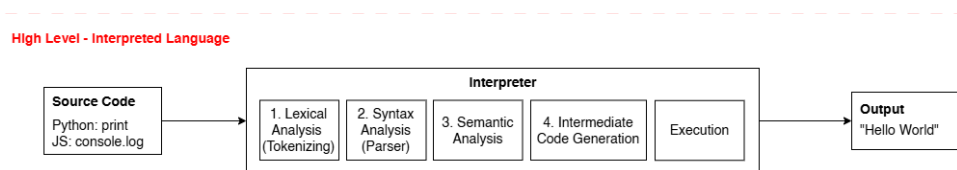
Daftar Isi.....	2
Dasar Teori.....	3
A. Intermediate Code.....	3
B. Intermediate Code Generation.....	4
a. Konvensi Mesin Stack.....	4
b. Translasi Alur Kontrol (Control Flow).....	5
C. Arsitektur Mesin Eksekusi: Interpreter & The Stack.....	5
a. Konsep Virtual Machine dan Siklus Eksekusi.....	5
b. Memori Eksekusi: The Stack.....	6
c. Runtime Type Checking.....	8
Perancangan dan Implementasi.....	9
A. Perancangan.....	9
B. Implementasi.....	9
a. IntermediateCode.....	9
b. IntermediateCodeGeneration.....	9
c. Interpreter.....	9
Pengujian.....	10
Kesimpulan dan Saran.....	12
A. Kesimpulan.....	12
B. Saran.....	12
Lampiran.....	13
Referensi.....	14

Dasar Teori

A. Intermediate Code

Dalam perancangan arsitektur kompilator modern, sistem secara umum dibagi ke dalam dua fase operasional utama, yaitu fase *Front-end* dan fase *Back-end*. Fase *Front-end* memiliki tanggung jawab untuk memproses teks kode sumber secara langsung, mulai dari tokenisasi, validasi tata bahasa, hingga pemeriksaan semantik, yang pada akhirnya menghasilkan struktur data murni berupa *Decorated Abstract Syntax Tree* (DAST). Setelah tahapan tersebut selesai, sistem memasuki fase *Back-end* yang sepenuhnya independen dari aturan sintaksis bahasa tingkat tinggi dan murni berfokus pada tahapan sintesis serta komputasi eksekusi.

Untuk memvisualisasikan bagaimana aliran program bergerak dari bentuk teks mentah pada fase *Front-end* hingga dieksekusi oleh mesin virtual pada fase *Back-end*, keseluruhan arsitektur kompilator dan *interpreter* bahasa Arion dapat dilihat pada bagan berikut:



Gambar 1. Proses Kompilasi dan Interpretasi Bahasa Arion dari Tokenisasi hingga Eksekusi

https://docs.google.com/document/d/1YHN6uFcagXI5AB17g_-7bCRnkVRqreR39kUc8u-2ax4/edit?tab=t.0#heading=h.7hjnc3sy3wu8

Sebagai jembatan translasi antara pemahaman konseptual pada fase *Front-end* dan proses eksekusi aktual pada fase *Back-end*, digunakanlah sebuah representasi perantara yang disebut sebagai *Intermediate Code*. *Intermediate Code* dibentuk dengan cara meratakan (*flattening*) struktur pohon hierarkis DAST yang rumit dan bersarang, menjadi sebuah daftar instruksi sekuensial yang linier. Representasi instruksi perantara ini didesain agar sangat sederhana dan menyerupai bahasa mesin (*machine code*), namun tetap dalam format yang cukup representatif untuk dibaca atau dianalisis oleh manusia.

Pemisahan arsitektur yang tegas melalui penggunaan *Intermediate Code* ini krusial untuk menciptakan sistem kompilator yang modular. Dengan arsitektur ini, modifikasi atau pengembangan sintaksis bahasa pemrograman baru di masa depan hanya akan menuntut perombakan pada komponen *Front-end*. Selama fase *Front-end* tersebut tetap mampu memproduksi format DAST yang dijanjikan, maka mesin *Back-end* akan selalu dapat menerima, mensintesis *Intermediate Code*, dan mengeksekusinya secara konsisten tanpa perlu mengalami modifikasi.

B. Intermediate Code Generation

Tahapan *Intermediate Code Generation* (ICG) merupakan proses krusial di mana representasi konseptual kode, seperti *Decorated Abstract Syntax Tree* (DAST), ditranslasikan menjadi format instruksi operasional. Pada fase ini, struktur pohon yang bersifat hierarkis dan bersarang akan diratakan (*flattening*) menjadi sebuah daftar instruksi sekuensial yang linier. DAST berfungsi sebagai sumber rujukan utama karena memuat anotasi semantik esensial, seperti tipe data dan referensi tabel simbol, yang diperlukan oleh generator untuk memutuskan jenis instruksi komputasi yang tepat.

a. Konvensi Mesin Stack

Dalam perancangan *Intermediate Code*, terdapat berbagai macam arsitektur representasi. Meskipun kompilator secara umum sering menggunakan *Three-Address Code* (TAC) yang mengandalkan alokasi variabel sementara, terdapat alternatif arsitektur lain yang lebih primitif yaitu instruksi mesin berbasis tumpukan (*Stack Machine*).

Berbeda dengan TAC yang membutuhkan register atau alamat spesifik untuk menyimpan hasil perhitungan sementara, *Stack Machine* beroperasi tanpa variabel penampung bernama. Arsitektur ini murni memanipulasi nilai pada pucuk tumpukan memori menggunakan prinsip *Last In, First Out* (LIFO). Evaluasi ekspresi dilakukan melalui instruksi yang secara implisit mengambil (*pop*) operan dari tumpukan teratas, menjalankan operasi komputasinya, lalu mengembalikan (*push*) hasil akhir tersebut kembali ke posisi teratas stack. Pendekatan ini menghasilkan representasi instruksi yang lebih ringkas dan evaluasi memori yang lebih tersentralisasi.

b. Translasi Alur Kontrol (Control Flow)

Translasi alur kontrol merupakan aspek paling kompleks dalam ICG, mengingat blok logika tingkat tinggi seperti percabangan (`IF-ELSE`) dan perulangan (`WHILE`) memiliki sifat bersarang dan tidak linear. Mesin komputer atau mesin virtual tidak dapat mengeksekusi struktur bersarang secara langsung. Oleh karena itu, struktur tersebut harus diratakan menggunakan kombinasi label target alamat dan instruksi pelompatan alur eksekusi. Terdapat dua jenis instruksi pelompatan utama, yaitu lompatan mutlak (*Unconditional Jump*) dan lompatan bersyarat (*Conditional Jump*).

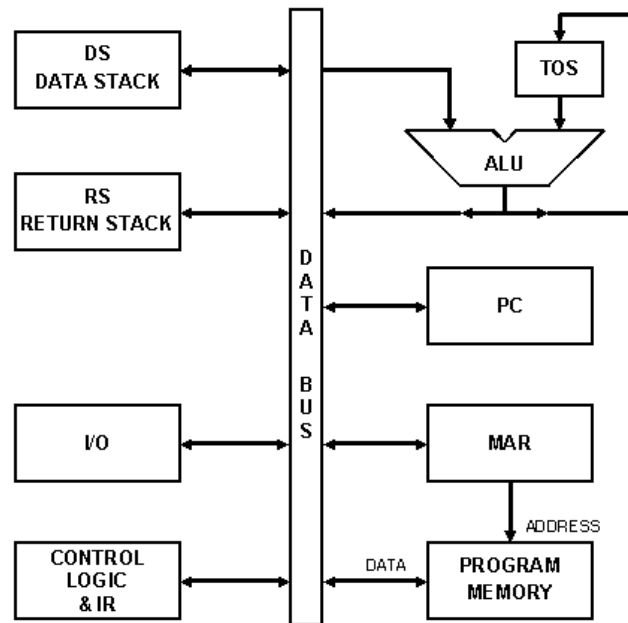
Dalam penerapannya di tingkat bahasa mesin, evaluasi kondisi bersyarat sering kali diimplementasikan menggunakan pendekatan pelompatan saat salah (`IF_FALSE`). Arsitektur ini mengeksploitasi mekanisme fall-through: apabila suatu kondisi bernilai benar, eksekusi instruksi dibiarkan mengalir secara alamiah ke baris komputasi berikutnya tanpa memerlukan intervensi lompatan sama sekali. Lompatan instruksi hanya akan dipicu apabila kondisi bernilai salah, guna mengalihkan penunjuk instruksi ke blok alternatif atau akhir blok. Pendekatan evaluasi lawan ini secara signifikan menyederhanakan topologi kode yang dihasilkan.

C. Arsitektur Mesin Eksekusi: Interpreter & The Stack

Tahapan akhir dari arsitektur kompilator dan eksekusi program direalisasikan melalui komponen *Interpreter*. Setelah *Intermediate Code* selesai disintesis, *Interpreter* mengambil alih peran eksekusi dengan bertindak sebagai sebuah Mesin Virtual (*Virtual Machine*). Mesin ini dirancang untuk mereplikasi logika operasional dan siklus pengolahan memori layaknya perangkat keras komputasi fisik, namun diimplementasikan sepenuhnya secara perangkat lunak.

a. Konsep Virtual Machine dan Siklus Eksekusi

Operasional *Virtual Machine* (VM) dikendalikan oleh sebuah penunjuk instruksi yang dikenal sebagai *Instruction Pointer* (IP) atau *Program Counter* (PC). Penunjuk ini berfungsi melacak baris instruksi aktual yang sedang dievaluasi dari daftar *Intermediate Code*.



Gambar 2. Arsitektur Komponen Internal dan Mekanisme Operasi pada Virtual Machine Berbasis Stack

https://users.ece.cmu.edu/~koopman/stack_computers/sec3_2.html

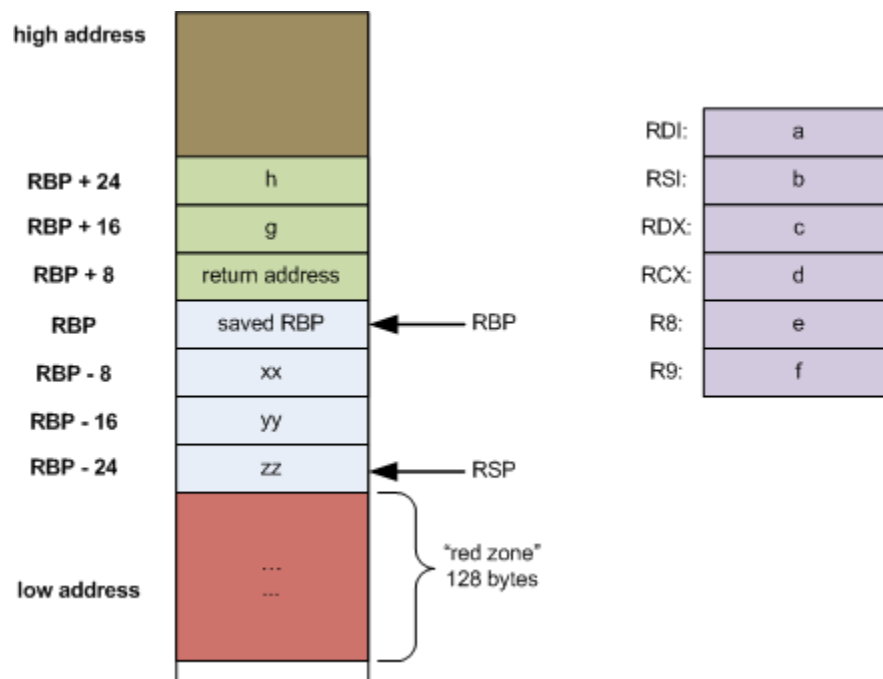
Proses eksekusi program pada VM beroperasi secara iteratif melalui tiga tahapan utama yang dikenal sebagai siklus *Fetch-Decode-Execute*. Pada tahap *Fetch*, VM membaca baris instruksi secara sekuensial berdasarkan posisi yang ditunjuk oleh IP. Selanjutnya pada tahap *Decode*, VM menguraikan instruksi tersebut guna memvalidasi perintah operasi dasar dan parameter operan yang menyertainya. Terakhir, tahap *Execute* memicu komputasi aktual dari perintah tersebut, baik berupa perhitungan matematis maupun manipulasi penyimpanan ke dalam memori. Setelah eksekusi suatu baris selesai, nilai IP umumnya akan diinkrementasi menuju baris instruksi berikutnya, terkecuali apabila instruksi tersebut merupakan perintah pelompatan (*jump*) yang akan mengubah nilai IP secara mutlak menuju alamat label spesifik. Siklus ini terus berulang hingga IP mencapai instruksi akhir atau terminasi program.

b. Memori Eksekusi: The Stack

Untuk mendukung komputasi dan manajemen variabel secara dinamis, arsitektur VM mengimplementasikan struktur memori berbasis *Stack* yang beroperasi dengan prinsip LIFO (*Last In, First Out*). Pemilihan struktur ini tidak lepas dari kebutuhan perancangan

sistem akan efisiensi alokasi memori terkait manajemen ruang lingkup (*scope*) dan mekanisme pemanggilan fungsi.

Setiap kali alur eksekusi memasuki blok kode atau fungsi baru, VM secara otomatis mengalokasikan sebuah ruang memori terisolasi yang disebut *Stack Frame* pada pucuk tumpukan memori. *Stack Frame* ini bertugas untuk menampung seluruh variabel lokal, argumen fungsi, serta alamat pengembalian eksekusi (*return address*).



Gambar 3. Anatomi Struktur Stack Frame

<https://eli.thegreenplace.net/2011/09/06/stack-frame-layout-on-x86-64>

Saat fungsi sedang berjalan, VM dibatasi untuk hanya dapat beroperasi pada blok memori teratas ini. Melalui operasi alokasi saat blok diinisiasi dan dealokasi saat blok selesai dieksekusi, VM memastikan bahwa variabel lokal dihancurkan seketika saat ruang lingkup panggilannya usai. Mekanisme memori dinamis ini mengamankan isolasi data antar fungsi sekaligus menjaga konsumsi memori program agar senantiasa optimal.

c. Runtime Type Checking

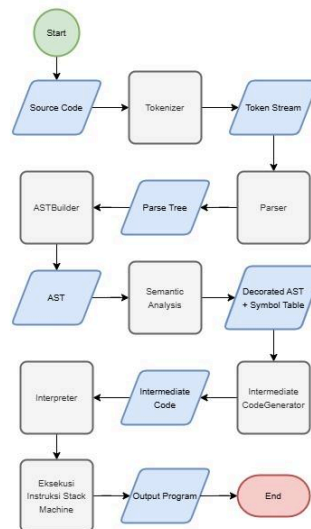
Dalam perancangan perangkat lunak *interpreter* secara umum, pemeriksaan keamanan tipe data saat program dieksekusi (*Runtime Type Checking*) merupakan pilar keamanan yang esensial, khususnya bagi arsitektur bahasa pemrograman yang bersifat dinamis (*weakly-typed*). Mekanisme pemeriksaan ini memvalidasi keabsahan operasi komputasi terhadap wujud tipe data aktual yang bermukim di memori guna mencegah interupsi atau ambiguitas eksekusi.

Meskipun demikian, pada perancangan kompilator Arion, mekanisme *Runtime Type Checking* tidak diwajibkan untuk diimplementasikan secara dinamis oleh *interpreter*. Justifikasi dari perancangan arsitektur ini dilandasi oleh karakteristik bahasa Arion yang dikategorikan sebagai bahasa *strongly-typed*. Validasi keamanan tipe data yang ketat, komprehensif, dan menyeluruh telah didelegasikan dan diselesaikan secara statis pada fase *Front-end*, yakni pada tahapan Analisis Semantik. Melalui pendekatan desain mendelegasikan pemeriksaan ke tahapan kompilator statis, mesin eksekusi dapat mengolah operasi secara konstan terbebas dari beban komputasi ganda (*overhead*), sehingga menghasilkan performa *runtime* yang cepat dan efisien.

Perancangan dan Implementasi

A. Perancangan

Pada milestone 4, sistem dirancang untuk melanjutkan hasil dari milestone sebelumnya, yaitu Decorated AST dan Symbol Table, menjadi Intermediate Code yang dapat dieksekusi. Intermediate Code yang digunakan berbentuk instruksi stack machine. Dengan pendekatan ini, setiap instruksi bekerja dengan memanfaatkan stack runtime, misalnya untuk menyimpan literal, mengambil nilai variabel, melakukan operasi aritmatika, melakukan percabangan, serta menjalankan procedure atau function.



Gambar 4. Diagram Flowchart Alur Besar Sistem

Alur sistem dimulai dari pembacaan source code, kemudian dilakukan tokenisasi oleh Tokenizer, parsing oleh Parser, pembentukan AST oleh ASTBuilder, dan analisis semantik oleh DecoratedAST. Hasil dari tahap tersebut berupa Decorated AST dan Symbol Table. Kedua struktur ini kemudian diberikan kepada IntermediateCodeGenerator untuk menghasilkan daftar instruksi Intermediate Code. Setelah itu, Interpreter mengeksekusi instruksi tersebut dan menghasilkan output program.

Komponen utama pada milestone ini terdiri dari `IntermediateCode`, `IntermediateCodeGenerator`, dan `Interpreter`. `IntermediateCode` bertugas menyimpan daftar instruksi stack machine dalam bentuk `InstructionLine`. `IntermediateCodeGenerator` bertugas menerjemahkan node-node pada `Decorated AST` menjadi instruksi stack machine dengan bantuan informasi dari `Symbol Table`, seperti alamat variabel, ukuran frame, level scope, serta metadata array dan record. Sementara itu, `Interpreter` bertugas mengeksekusi instruksi yang telah dihasilkan menggunakan stack runtime, instruction pointer, dan call frame.

Untuk merepresentasikan program dalam bentuk stack machine, setiap instruksi `Intermediate Code` disimpan sebagai satu `InstructionLine`. Setiap `InstructionLine` terdiri atas opcode, level, operand, dan komentar. Opcode menunjukkan jenis instruksi yang dijalankan, level digunakan untuk menunjukkan perbedaan lexical scope pada akses variabel atau pemanggilan subprogram, operand menyimpan nilai tambahan seperti alamat atau target lompatan, sedangkan komentar digunakan untuk membantu pembacaan hasil `Intermediate Code`. Daftar opcode yang digunakan pada implementasi ditunjukkan pada tabel berikut.

Daftar Instruksi			
No	Instruksi	Operasi	Keterangan
1	LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack
2	LOD a	Load Value	Memuat nilai dari address a
3	STO a	Store Value	Menyimpan nilai ke address a
4	CAL l	Call	Memanggil fungsi di garis l
5	INT m	Initiate Memory	Membuat memory dengan ukuran m
6	JMPI	Unconditional Jump	Lompat ke garis l tanpa

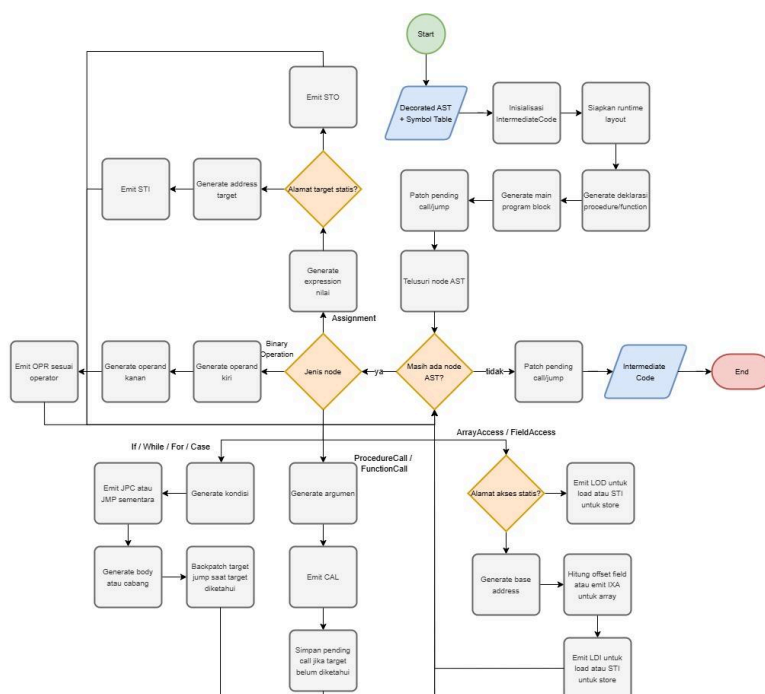
			kondisi apapun
7	JPC l	Conditional Jump	Lompat ke garis l bila kondisi tidak memenuhi
8	OPR o	Operation	Memanggil operasi o (Termasuk fungsi bawaan Arion)
9	LDA l a	Load Address	Memasukkan alamat variabel pada level l dan alamat relatif a ke dalam stack.
10	IXA low high size	Index Address	Menghitung alamat elemen array berdasarkan index yang ada di stack, batas bawah low, batas atas high, dan ukuran elemen size.
11	LDI	Load Indirect	Mengambil alamat dari stack, lalu memasukkan nilai yang berada pada alamat tersebut ke dalam stack.
12	STI	Store Indirect	Mengambil alamat dan nilai dari stack, lalu menyimpan nilai tersebut ke alamat yang ditunjuk.
13	RET	Return	Keluar dari fungsi atau prosedur

Beberapa operasi seperti penjumlahan, perbandingan, dan operasi keluaran tidak direpresentasikan sebagai opcode utama yang terpisah. Operasi tersebut direpresentasikan menggunakan instruksi OPR dengan operand berupa kode operasi. Dengan rancangan ini, instruksi utama tetap sederhana, sedangkan variasi operasi aritmatika, relasional, dan keluaran dapat dikelompokkan dalam satu opcode. Daftar kode operasi pada instruksi OPR ditunjukkan pada tabel berikut.

Instruksi OPR		
No	Operation Name	Function
1	NEG	Negasi value dari stack teratas
2	ADD	Menambah value stack teratas dengan value stack teratas satunya lagi
3	SUB	Mengurangi value stack teratas dengan value stack teratas satunya lagi
4	MUL	Mengali value stack teratas dengan value stack teratas satunya lagi
5	RDIV	Membagi dua nilai teratas sebagai bilangan real dan memasukkan hasil real ke stack.
6	MOD	Modulus value stack teratas dengan value stack teratas satunya lagi
7	EQL	Membandingkan kedua value teratas dari stack untuk melihat apakah kedua value tersebut sama (Conditionals)
8	NEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah kedua value tersebut tidak sama (Conditionals)
9	LSS	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama kurang dari value kedua (Conditionals)
10	GEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama lebih dari sama dengan value kedua (Conditionals)
11	GTR	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama lebih dari value kedua (Conditionals)
12	LEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama kurang dari sama dengan value kedua (Conditionals)

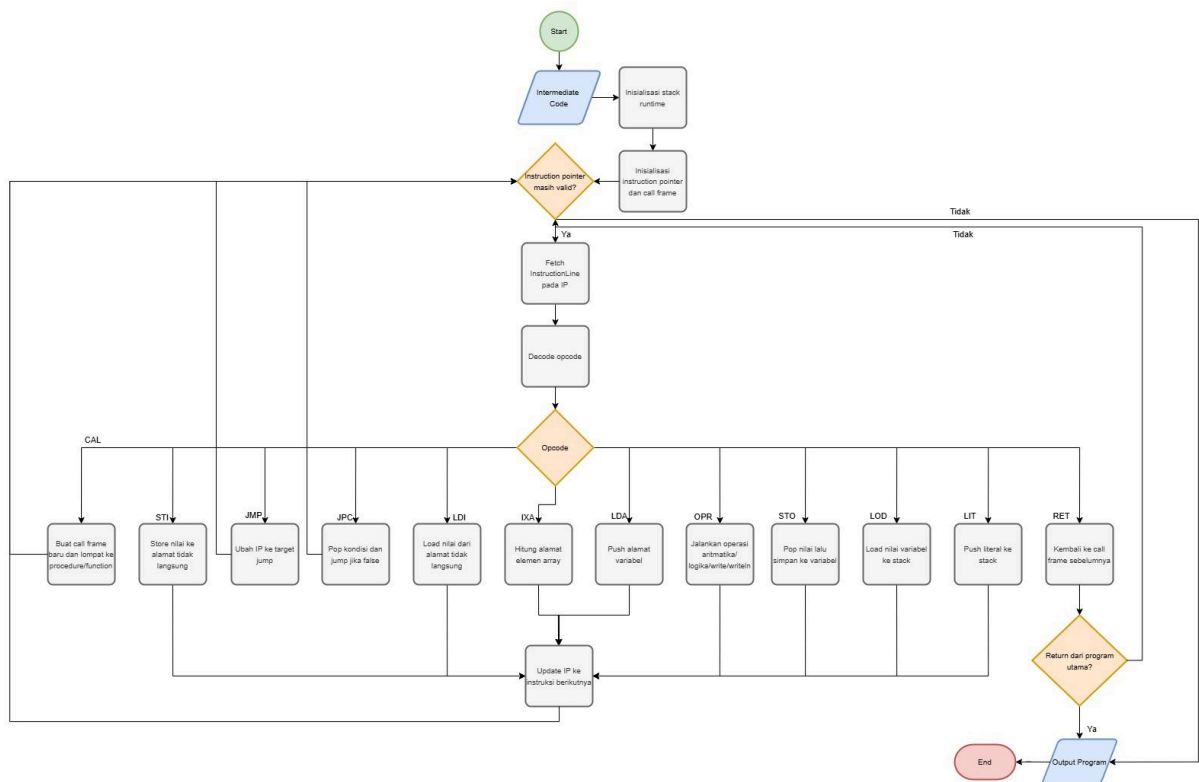
13	WRT	Menulis output yang sudah dimuat
14	WRTLN	Menulis output yang sudah dimuat lalu diberikan newline
15	IDIV	Membagi dua nilai integer teratas dan memasukkan hasil pembagian integer ke stack.
16	I2R	Mengonversi nilai integer pada stack teratas menjadi nilai real.

IntermediateCodeGenerator dirancang dengan pendekatan traversal terhadap Decorated AST. Setiap jenis node AST diterjemahkan menjadi satu atau lebih instruksi stack machine. Untuk assignment, generator terlebih dahulu menghasilkan instruksi untuk expression, kemudian menyimpan hasilnya ke target. Untuk operasi biner, generator menghasilkan instruksi operand kiri dan kanan, lalu menghasilkan instruksi OPR. Untuk percabangan dan perulangan, generator menggunakan instruksi JPC dan JMP, kemudian melakukan backpatching ketika target lompatan sudah diketahui.



Gambar 5. Diagram Flowchart Proses IntermediateCodeGenerator

Interpreter dirancang sebagai mesin stack sederhana. Interpreter membaca setiap InstructionLine berdasarkan instruction pointer, kemudian mengeksekusi instruksi sesuai opcode. Nilai sementara disimpan di stack runtime, sedangkan pemanggilan procedure dan function dikelola menggunakan call frame. Dengan rancangan ini, program hasil kompilasi dapat dijalankan langsung dari Intermediate Code tanpa perlu diterjemahkan terlebih dahulu ke bahasa mesin asli.



Gambar 6. Diagram Flowchart Proses Interpreter

Selain instruksi dasar yang terdapat pada spesifikasi, implementasi menambahkan beberapa instruksi untuk mendukung akses data yang lebih kompleks, terutama array dan record. Instruksi tambahan tersebut adalah LDA, IXA, LDI, dan STI. LDA digunakan untuk mengambil alamat suatu variabel, IXA digunakan untuk menghitung alamat elemen array berdasarkan index, LDI digunakan untuk mengambil nilai dari alamat yang dihitung saat runtime, dan STI digunakan untuk menyimpan nilai ke alamat yang dihitung saat runtime. Penambahan instruksi ini diperlukan karena akses seperti `arr[i]`

tidak selalu dapat ditentukan alamatnya pada compile time, sehingga alamat elemen harus dihitung saat program berjalan.

Implementasi juga memperluas operasi pada instruksi OPR dengan membedakan pembagian real dan pembagian integer melalui operasi RDIV dan IDIV. RDIV digunakan untuk operator / dan selalu menghasilkan nilai real, sedangkan IDIV digunakan untuk operator div dengan operand dan hasil bertipe integer. Selain itu, operasi I2R ditambahkan untuk mengonversi nilai integer menjadi real, misalnya ketika nilai integer diberikan kepada variabel, parameter, atau nilai kembalian bertipe real. Penambahan ini diperlukan agar aturan tipe dan pembagian tetap dipertahankan saat program dieksekusi.

B. Implementasi

Implementasi dilakukan dalam bahasa C++ dan dibuat dalam *namespace* "arion" agar terpisah dari *standard library* C++.

a. IntermediateCode

IntermediateCode adalah program hasil proses code generation yang akan dieksekusi oleh interpreter. Intermediate code tidak disimpan sebagai string mentah, tetapi direpresentasikan sebagai kumpulan objek InstructionLine yang berada di dalam `std::vector<InstructionLine> instructionLines_` pada kelas IntermediateCode. Rancangan ini memungkinkan generator menambahkan instruksi secara bertahap, menyimpan indeks instruksi sebagai alamat lompatan, serta memperbarui operand instruksi tertentu ketika target JMP, JPC, atau CAL baru dapat diketahui.

Setiap instruksi direpresentasikan oleh kelas InstructionLine yang memiliki empat komponen utama, yaitu opCode, level, operand, dan comment. Opcode didefinisikan melalui enum InstructionOpCode, sehingga instruksi utama yang dibutuhkan seperti LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, dan RET, tersedia dalam bentuk yang terstruktur. Selain instruksi dasar tersebut, implementasi juga menambahkan LDA, IXA, LDI, dan STI untuk menangani pembentukan alamat record dan array serta akses memori tidak langsung. Dengan demikian, representasi intermediate code tidak hanya mencakup model minimal stack machine, tetapi juga diperluas untuk mendukung struktur data.

Field level digunakan untuk menyatakan perbedaan lexical level pada instruksi seperti LOD, STO, CAL, dan LDA, sedangkan field operand disimpan sebagai std::string. Hal ini membuat satu struktur instruksi dapat digunakan untuk berbagai kebutuhan, seperti menyimpan literal pada LIT, alamat memori pada LOD dan STO, target lompatan pada JMP dan JPC, serta nomor operasi pada OPR. Untuk instruksi OPR, operasi seperti NEG, ADD, SUB, MUL, DIV, MOD, EQL, NEQ, LSS, GEQ, GTR, LEQ, WRT, dan WRTLN direpresentasikan melalui enum OperationCode, kemudian dikonversi menjadi bilangan menggunakan InstructionLine::operationToInt. Dengan demikian, format instruksi tetap sederhana, sementara detail operasi ditangani oleh interpreter.

Instruksi yang telah dibentuk dapat diformat menjadi keluaran teks dengan pola nomor baris, opcode, level, dan operand, disertai komentar opsional dalam tanda kurung kurawal. Kelas IntermediateCode menyediakan beberapa helper, seperti emitLIT, emitLOD, emitSTO, emitCAL, emitINT, emitJMP, emitJPC, emitOPR, dan emitRET. Helper tersebut membuat generator tidak perlu menyusun objek instruksi secara manual setiap kali menerbitkan instruksi baru. Selain itu, metode patchInstructionOperand digunakan untuk melakukan backpatching terhadap target lompatan atau alamat pemanggilan setelah posisi instruksi tujuan diketahui. Rancangan ini memberi antarmuka yang konsisten bagi generator dan tetap menghasilkan struktur instruksi yang mudah dibaca kembali oleh interpreter melalui getInstructionLine.

Berikut adalah definisi struktur data IntermediateCode pada src/arion/IntermediateCode.hpp:

```
namespace arion {  
    enum class InstructionOpCode {  
        LIT,  
        LOD,  
        STO,  
        CAL,  
        INT,  
        JMP,
```



```

        JPC,
        OPR,
        LDA,
        IXA,
        LDI,
        STI,
        RET
};

enum class OperationCode {
    NEG = 1,
    ADD = 2,
    SUB = 3,
    MUL = 4,
    DIV = 5,
    MOD = 6,
    EQL = 7,
    NEQ = 8,
    LSS = 9,
    GEQ = 10,
    GTR = 11,
    LEQ = 12,
    WRT = 13,
    WRTLN = 14
};

class InstructionLine {
public:
    InstructionOpCode opCode = InstructionOpCode::LIT;
    int level = 0;
    std::string operand;
    std::string comment;

    std::string toString(std::size_t lineNumber)
const;
    static std::string
opCodeToString(InstructionOpCode opCode);
    static int operationToInt(OperationCode
operation);
};

class IntermediateCode {
private:
    std::vector<InstructionLine> instructionLines_;

```

```

    public:
        std::size_t getTotalLine() const;
        const InstructionLine *getInstructionLine(int
position) const;
        void printCode(std::ostream &out = std::cout)
const;

        std::size_t getNextInstructionIndex() const;
        std::size_t emitInstruction(InstructionOpCode
opCode, int level = 0, std::string operand = "",
std::string comment = "");
        std::size_t emitLIT(std::string value, std::string
comment = "");
        std::size_t emitLOD(int address, std::string
comment = "");
        std::size_t emitLOD(int level, int address,
std::string comment = "");
        std::size_t emitSTO(int address, std::string
comment = "");
        std::size_t emitSTO(int level, int address,
std::string comment = "");
        std::size_t emitCAL(int line, std::string comment
= "");
        std::size_t emitCAL(int level, int line,
std::string comment = "");
        std::size_t emitINT(int size, std::string comment
= "");
        std::size_t emitJMP(int line, std::string comment
= "");
        std::size_t emitJPC(int line, std::string comment
= "");
        std::size_t emitOPR(OperationCode operation,
std::string comment = "");
        std::size_t emitLDA(int level, int address,
std::string comment = "");
        std::size_t emitIXA(int low, int high, int
elementSize, std::string comment = "");
        std::size_t emitLDI(std::string comment = "");
        std::size_t emitSTI(std::string comment = "");
        std::size_t emitRET(std::string comment = "");
        void patchInstructionOperand(std::size_t line, int
operand);
    };

```

}

b. IntermediateCodeGenerator

Program `IntermediateCodeGenerator` berfungsi untuk menerjemahkan `Decorated AST` menjadi daftar instruksi `intermediate code`. Masukan program adalah `ASTNode` yang telah didekorasi oleh tahap `semantic analysis`, sehingga setiap node sudah membawa informasi semantik melalui `ASTAnnotation`, seperti `tabIndex`, `blockIndex`, `arrayIndex`, dan `lexicalLevel`. Generator juga menerima `SymbolTable` melalui konstruktor. Dengan demikian, proses pembangkitan kode memanfaatkan struktur pohon `AST` sekaligus metadata simbol dari `tab`, `btabs`, `atab`, dan `descriptor tipe`.

Proses utama dimulai dari pemanggilan `generate(const ASTNode &decoratedAst)`. Pada tahap ini, generator mereset state, menyiapkan `layout runtime` melalui `prepareRuntimeLayout`, memanggil `generateProgram`, lalu menyelesaikan pemanggilan yang masih tertunda melalui `patchPendingCalls`. Generator terlebih dahulu memetakan entri simbol ke alamat `runtime` melalui `tabRuntimeAddress_`, mencatat block pemilik simbol melalui `tabOwnerBlock_`, menghitung ukuran frame setiap `activation block` melalui `frameSizes_`, serta menyiapkan slot nilai balik fungsi melalui `functionReturnAddress_`. Parameter ditempatkan mulai alamat 3 karena tiga slot pertama frame digunakan untuk `static link`, `dynamic link`, dan `return address`. Variabel lokal ditempatkan setelah parameter, sedangkan block anonim dipetakan ke frame aktivasi yang sama melalui `mapBlockStorage`.

Metode `generateNode` membedakan node kontainer, seperti `Program`, `Block`, `CompoundStatement`, dan `StatementList`, dari node `statement` dan `expression`. Deklarasi konstanta, tipe, dan variabel tidak menghasilkan instruksi langsung karena kebutuhan `runtime`-nya sudah dihitung pada tahap `layout`. Pada program utama, `generateProgram` terlebih dahulu menempatkan instruksi `JMP` menuju entry program utama agar kode subprogram dapat ditempatkan sebelum badan utama. Setelah deklarasi `procedure` dan

function diterjemahkan melalui `generateSubprogramDeclarations`, target `JMP` tersebut diperbarui ke posisi awal main program. Generator kemudian menerbitkan `INT` untuk inisialisasi frame program, menerjemahkan statement utama, dan menutup program dengan `RET`.

Pada translasi subprogram, generator mencatat baris entri setiap subprogram di dalam `subprogramEntryLines_`. Setiap subprogram diawali dengan `JMP` internal untuk melewati kemungkinan deklarasi subprogram bersarang sebelum memasuki badan efektifnya. Setelah itu, generator menerbitkan `INT` sesuai ukuran frame subprogram. Jika entri simbol menunjukkan bahwa subprogram tersebut adalah fungsi, maka sebelum `RET` generator memuat kembali nilai dari slot hasil fungsi menggunakan `LOD`.

Pada statement assignment, generator mengevaluasi sisi kanan terlebih dahulu melalui `generateExpression`, kemudian menyimpan hasilnya ke target menggunakan `generateVariableStore`. Untuk target sederhana, generator cukup meng-emit `STO`. Untuk target kompleks, seperti array dinamis atau field record yang membutuhkan dereferensi, generator mencari alamat target terlebih dahulu dan menutup proses penyimpanan dengan `STI`.

Translasi ekspresi dilakukan melalui `generateExpression`. Literal bertipe integer, real, char, string, dan boolean diterjemahkan menjadi instruksi `LIT`. Identifier dan variabel biasa diterjemahkan menjadi `LOD` jika alamatnya dapat ditentukan secara statis. Jika node menunjuk konstanta, generator tidak menggunakan `LOD`, tetapi langsung emit `LIT` berdasarkan nilai konstanta tersebut. Untuk assignment terhadap nama fungsi di dalam badan fungsi, generator memperlakukannya sebagai penyimpanan ke slot hasil fungsi melalui `functionReturnAddress_`, sehingga pola Pascal-like seperti `FunctionName := value` dapat direpresentasikan.

Ekspresi unary dan binary dipetakan ke instruksi `OPR`. Operator aritmetika `+`, `-`, `*`, `/`, `div`, dan `mod` dipetakan masing-masing ke `ADD`, `SUB`, `MUL`, `DIV`, dan `MOD`. Operator

relasional =, <, <=, >, dan >= dipetakan ke EQL, NEQ, LSS, GEQ, GTR, dan LEQ. Unary minus diterjemahkan menjadi NEG. Untuk operator boolean, implementasi menggunakan pendekatan aritmetika stack: and diterjemahkan sebagai MUL, or sebagai ADD, kemudian hasilnya dinormalisasi menjadi boolean melalui perbandingan terhadap false menggunakan NEQ. Operator not diterjemahkan dengan membandingkan operand terhadap false menggunakan EQL.

Akses variabel komposit ditangani dengan dua mekanisme. Jika alamat dapat dihitung secara statis, misalnya field record dengan offset tetap atau array konstan, generator langsung menggunakan LOD atau STO pada alamat relatif terhadap frame. Namun, jika alamat harus dihitung saat runtime, generator menggunakan generateVariableAddress untuk menerbitkan LDA. Untuk array dinamis, generator menambahkan IXA dengan informasi batas bawah, batas atas, dan ukuran elemen. Setelah alamat efektif berada di stack, pembacaan dilakukan dengan LDI, sedangkan penulisan dilakukan dengan STI.

Pemanggilan write dan writeln diperlakukan sebagai kasus khusus. Generator tidak menerjemahkannya menjadi CAL, tetapi langsung mengevaluasi setiap argumen, kemudian menerbitkan OPR WRT atau OPR WRTLN. Untuk writeln tanpa argumen, generator menerbitkan literal string kosong sebelum WRTLN agar newline tetap dihasilkan. Sebaliknya, pemanggilan procedure dan function biasa diterjemahkan dengan mengevaluasi semua argumen, menempatkan jumlah argumen melalui LIT, lalu menerbitkan CAL. Jika target subprogram belum diketahui ketika CAL dibentuk, generator menyimpannya dalam pendingCallPatches_ dan memperbarui target tersebut pada tahap patchPendingCalls.

Percabangan dan perulangan diterjemahkan menggunakan pola umum stack machine. Statement if diterjemahkan dengan mengevaluasi kondisi, menerbitkan JPC ke cabang else atau akhir blok, menjalankan badan then, dan menambahkan JMP apabila terdapat cabang else. Statement while diterjemahkan dengan label awal loop, evaluasi kondisi,

JPC ke akhir loop, badan loop, lalu JMP kembali ke awal. Statement repeat-until dimulai dari badan loop, dilanjutkan dengan evaluasi kondisi, lalu JPC kembali ke awal selama kondisi masih bernilai salah. Statement for diterjemahkan dengan assignment nilai awal, evaluasi kondisi menggunakan LEQ untuk to atau GEQ untuk downto, JPC ke akhir loop, badan loop, pembaruan variabel kontrol dengan ADD atau SUB terhadap nilai 1, lalu JMP kembali ke awal. Statement case juga diterjemahkan dengan membandingkan setiap label terhadap ekspresi selector menggunakan EQL, kemudian menggunakan JPC dan JMP untuk mengatur perpindahan antar-branch serta mencegah fall-through setelah sebuah branch selesai.

Berikut adalah definisi struktur data IntermediateCodeGenerator pada src/arion/IntermediateCodeGenerator.hpp:

```
namespace arion {
    class IntermediateCodeGenerator {
    public:
        explicit IntermediateCodeGenerator(const
SymbolTable &symbolTable);

        IntermediateCode generate(const ASTNode
&decoratedAst);

    private:
        const SymbolTable &symbolTable_;
        IntermediateCode code_;
        int currentBlockIndex_ = 0;
        std::unordered_map<int, int> tabOwnerBlock_;
        std::unordered_map<int, int> tabRuntimeAddress_;
        std::unordered_map<int, int> frameSizes_;
        std::unordered_map<int, int>
functionReturnAddress_;
        std::unordered_map<int, int>
subprogramEntryLines_;
        std::vector<std::pair<std::size_t, int>>
pendingCallPatches_;

        void generateProgram(const ASTNode &node);
        void generateSubprogramDeclarations(const ASTNode
```

```

&node);

    void generateSubprogram(const ASTNode &node);
    void generateNode(const ASTNode &node);
    void generateStatement(const ASTNode &node);
    void generateAssignment(const ASTNode &node);
    void generateIfStatement(const ASTNode &node);
    void generateCaseStatement(const ASTNode &node);
    void generateWhileStatement(const ASTNode &node);
    void generateRepeatStatement(const ASTNode &node);
    void generateForStatement(const ASTNode &node);
    void generateProcedureCall(const ASTNode &node);
    void generateFunctionCall(const ASTNode &node);
    void generateExpression(const ASTNode &node);
    void generateVariableLoad(const ASTNode &node);
    void generateVariableStore(const ASTNode &node);
    void generateVariableAddress(const ASTNode &node);

    void prepareRuntimeLayout();
    void mapBlockStorage(int blockIndex, int
&nextAddress);
    void patchPendingCalls();
    std::vector<int> tabEntriesForBlock(int
blockIndex) const;
    std::vector<const ASTNode *> callArguments(const
ASTNode &node) const;
    int frameSize(const ASTNode &programNode) const;
    int frameSizeForBlock(int blockIndex) const;
    int nodeTabIndex(const ASTNode &node) const;
    int nodeAddress(const ASTNode &node) const;
    std::pair<int, int> variableAccessAddress(const
ASTNode &node) const;
    bool fixedVariableAccessAddress(const ASTNode
&node, int &level, int &address) const;
    std::pair<TypeKind, int> nodeValueType(const
ASTNode &node) const;
    int fieldTabIndex(const ASTNode &node) const;
    bool staticIndexOrdinal(const ASTNode &node, int
&ordinal) const;
    bool literalOrdinal(TypeKind type, const
std::string &value, int &ordinal) const;
    int tabAddress(int tabIndex) const;
    int tabBlockIndex(int tabIndex) const;
    int activationBlockIndex(int blockIndex) const;
    int lexicalLevelDiffToBlock(int

```

```

targetActivationBlockIndex) const;
    int lexicalLevelDiffForTab(int tabIndex) const;
    int callableLevelDiff(int calleeBlockIndex) const;
    int lookupVisibleTabIndex(const std::string &name)
const;
    int emitCall(const TabEntry &entry, const
std::string &name, std::size_t argumentCount);
    std::string literalValue(const ASTNode &node)
const;
    OperationCode binaryOperation(const ASTNode &node)
const;
    std::string operatorText(const ASTNode &node)
const;
    std::string normalizeIdentifier(const std::string
&identifier) const;
    bool sameIdentifier(const std::string &left, const
std::string &right) const;
    bool isWriteProcedure(const ASTNode &node) const;
    bool isWriteLnProcedure(const ASTNode &node)
const;
    bool isActivationBlock(BlockKind kind) const;
    bool isFunctionReturnTarget(int tabIndex) const;
    const ASTNode *requiredChild(const ASTNode &node,
ASTChildRole role) const;
};

class IntermediateCodeGeneratorError : public
std::runtime_error {
public:
    explicit IntermediateCodeGeneratorError(const
std::string &message);
};
}

```

c. Interpreter

Interpreter merupakan komponen eksekusi akhir pada Milestone 4 yang bertugas menjalankan Intermediate Code berbentuk instruksi stack machine. Komponen ini menerima objek IntermediateCode, membaca setiap InstructionLine berdasarkan instruction pointer, kemudian mengeksekusi instruksi tersebut hingga program selesai atau terjadi runtime error. Implementasi interpreter berada pada src/arion/Interpreter.hpp dan src/arion/Interpreter.cpp.

Antarmuka utama interpreter terdiri atas kelas `RuntimeError` dan kelas `Interpreter`. `RuntimeError` digunakan untuk melaporkan kesalahan yang terjadi saat program berjalan, seperti stack underflow, alamat memori tidak valid, pembagian dengan nol, akses array di luar batas, dan penggunaan nilai yang belum diinisialisasi. Kelas `Interpreter` memiliki konstruktor dengan `instructionLimit` untuk mencegah program berjalan tanpa henti, serta method `execute` dan `executeToString`. Method `execute` menjalankan intermediate code dan menulis keluaran program ke ostream, sedangkan `executeToString` digunakan untuk menjalankan program dan mengambil output-nya sebagai string.

Secara internal, interpreter menggunakan struktur `RuntimeValue` untuk merepresentasikan nilai runtime. `RuntimeValue` menyimpan jenis nilai melalui `ValueKind`, yaitu `Uninitialized`, `Integer`, `Real`, `Boolean`, dan `String`. Dengan rancangan ini, stack dapat menyimpan berbagai jenis nilai tanpa memerlukan stack terpisah untuk setiap tipe data. Literal yang berasal dari instruksi LIT diparsing menjadi `RuntimeValue` sesuai bentuknya, misalnya bilangan bulat menjadi `Integer`, bilangan berkoma menjadi `Real`, `true` dan `false` menjadi `Boolean`, serta literal karakter atau string menjadi `String`.

Selain runtime value, interpreter juga menggunakan struktur `FrameInfo` untuk menyimpan informasi activation record atau stack frame. Setiap frame menyimpan base address, ukuran frame, dan return address. Tiga slot pertama pada frame digunakan sebagai static link, dynamic link, dan return address. Static link digunakan untuk mengakses variabel pada lexical scope luar, dynamic link digunakan untuk mengetahui frame pemanggil, sedangkan return address digunakan untuk kembali ke instruksi setelah pemanggilan procedure atau function selesai.

Eksekusi program dilakukan melalui siklus fetch-decode-execute. Pada setiap iterasi, interpreter mengambil instruksi berdasarkan nilai instruction pointer, memeriksa opcode, lalu menjalankan aksi yang sesuai. Instruksi `INT` digunakan untuk mengalokasikan ukuran frame. Instruksi `LIT` memasukkan literal ke stack. Instruksi `LOD`

mengambil nilai dari alamat relatif terhadap frame aktif, sedangkan STO menyimpan nilai ke alamat tersebut. Untuk akses tidak langsung, LDA memasukkan alamat efektif ke stack, IXA menghitung alamat elemen array berdasarkan indeks dan batas array, LDI mengambil nilai dari alamat yang dihitung, dan STI menyimpan nilai ke alamat tidak langsung tersebut.

Instruksi JMP dan JPC digunakan untuk mengatur alur kontrol. JMP mengubah instruction pointer secara langsung menuju target tertentu, sedangkan JPC hanya melompat apabila nilai kondisi pada puncak stack bernilai false. Dengan mekanisme ini, struktur tingkat tinggi seperti if, while, repeat-until, for, dan case dapat dieksekusi sebagai urutan instruksi linear.

Pemanggilan procedure dan function ditangani melalui instruksi CAL. Sebelum CAL dieksekusi, jumlah argumen diletakkan di stack oleh code generator. Interpreter kemudian mengambil jumlah argumen tersebut, memindahkan argumen ke frame baru, membuat static link, dynamic link, dan return address, lalu mengubah instruction pointer menuju alamat awal subprogram. Ketika instruksi RET dijalankan, interpreter menghapus frame aktif, mengembalikan instruction pointer ke return address, dan apabila subprogram adalah function, nilai hasil fungsi tetap dipertahankan di stack agar dapat digunakan oleh caller.

Instruksi OPR digunakan untuk operasi aritmatika, relasional, unary, dan output. Operasi ADD, SUB, MUL, DIV, dan MOD mengambil dua operand teratas dari stack, melakukan operasi, lalu memasukkan hasilnya kembali ke stack. Operasi pembagian dengan nol akan menghasilkan RuntimeError. Untuk operasi perbandingan seperti EQL, NEQ, LSS, GEQ, GTR, dan LEQ, interpreter membandingkan dua operand teratas dan menghasilkan nilai Boolean. Operasi WRT dan WRTLN digunakan untuk menulis nilai ke output program, dengan WRTLN menambahkan newline setelah nilai ditulis.

Interpreter juga dilengkapi pemeriksaan runtime defensif. Walaupun validasi tipe utama telah dilakukan pada tahap semantic analysis, interpreter tetap memeriksa kondisi eksekusi yang hanya dapat diketahui saat runtime, seperti stack kosong, instruksi tidak valid, target jump di luar rentang, alamat memori tidak valid, nilai belum diinisialisasi, pembagian dengan nol, dan indeks array di luar batas. Dengan demikian, program yang gagal saat runtime tidak menyebabkan crash pada interpreter, tetapi menghasilkan pesan error yang informatif.

Pengujian

Kasus uji dapat dilihat di bagan berikut dan folder test/milestone-3/ di repositori GitHub

Tabel Hasil Pengujian	
1.	Input <pre> 1 program ArithmeticControl; 2 3 ∨ var 4 i, sum, countdown, branchValue: integer; 5 ratio: real; 6 ok: boolean; 7 8 ∨ begin 9 sum := 0; 10 for i := 1 to 4 do 11 ∨ begin 12 sum := sum + i 13 end; 14 15 countdown := 3; 16 while countdown > 0 do 17 ∨ begin 18 sum := sum + countdown; 19 countdown := countdown - 1 20 end; 21 </pre>

```
22  ✓ repeat
23    | sum := sum - 1
24    until sum == 15;
25
26    ratio := 5 / 2;
27    ok := (sum == 15) and not false or false;
28
29    writeln('sum = ', sum);
30    writeln('5 / 2 = ', ratio);
31    writeln('5 div 2 = ', 5 div 2);
32    writeln('5 mod 2 = ', 5 mod 2);
33    writeln('logic = ', ok);
34
35  ✓ case sum of
36    | 10: branchValue := 1;
37    | 15: branchValue := 2;
38    | 20: branchValue := 3
39    end;
40    writeln('case branch = ', branchValue)
41  end.
```

Output Intermediate Code

```
1  0 JMP 0 1 { program entry }
2  1 INT 0 9 { init program frame }
3  2 LIT 0 0 { literal }
4  3 STO 0 4 { store Variable }
5  4 LIT 0 1 { literal }
6  5 STO 0 3 { store Identifier }
7  6 LOD 0 3 { load Identifier }
8  7 LIT 0 4 { literal }
9  8 OPR 0 12 { for condition }
10 9 JPC 0 19 { for false }
11 10 LOD 0 4 { load Variable }
12 11 LOD 0 3 { load Variable }
13 12 OPR 0 2 { binary + }
14 13 STO 0 4 { store Variable }
15 14 LOD 0 3 { load Identifier }
16 15 LIT 0 1 { for step }
17 16 OPR 0 2 { for update }
18 17 STO 0 3 { store Identifier }
19 18 JMP 0 6 { repeat for }
20 19 LIT 0 3 { literal }
```

```
21    20 STO 0 5 { store Variable }
22    21 LOD 0 5 { load Variable }
23    22 LIT 0 0 { literal }
24    23 OPR 0 11 { binary > }
25    24 JPC 0 34 { while false }
26    25 LOD 0 4 { load Variable }
27    26 LOD 0 5 { load Variable }
28    27 OPR 0 2 { binary + }
29    28 STO 0 4 { store Variable }
30    29 LOD 0 5 { load Variable }
31    30 LIT 0 1 { literal }
32    31 OPR 0 3 { binary - }
33    32 STO 0 5 { store Variable }
34    33 JMP 0 21 { repeat while }
35    34 LOD 0 4 { load Variable }
36    35 LIT 0 1 { literal }
37    36 OPR 0 3 { binary - }
38    37 STO 0 4 { store Variable }
39    38 LOD 0 4 { load Variable }
40    39 LIT 0 15 { literal }
```

```
41 40 OPR 0 7 { binary == }
42 41 JPC 0 34 { repeat until false }
43 42 LIT 0 5 { literal }
44 43 LIT 0 2 { literal }
45 44 OPR 0 5 { binary / }
46 45 STO 0 7 { store Variable }
47 46 LOD 0 4 { load Variable }
48 47 LIT 0 15 { literal }
49 48 OPR 0 7 { binary == }
50 49 LIT 0 false { literal }
51 50 LIT 0 false { not compare }
52 51 OPR 0 7 { not }
53 52 OPR 0 4 { binary and }
54 53 LIT 0 false { normalize boolean }
55 54 OPR 0 8 { boolean result }
56 55 LIT 0 false { literal }
57 56 OPR 0 2 { binary or }
58 57 LIT 0 false { normalize boolean }
59 58 OPR 0 8 { boolean result }
60 59 STO 0 8 { store Variable }
```



```
61    60 LIT 0 'sum = ' { literal }
62    61 OPR 0 13 { write }
63    62 LOD 0 4 { load Variable }
64    63 OPR 0 14 { writeln }
65    64 LIT 0 '5 / 2 = ' { literal }
66    65 OPR 0 13 { write }
67    66 LOD 0 7 { load Variable }
68    67 OPR 0 14 { writeln }
69    68 LIT 0 '5 div 2 = ' { literal }
70    69 OPR 0 13 { write }
71    70 LIT 0 5 { literal }
72    71 LIT 0 2 { literal }
73    72 OPR 0 15 { binary div }
74    73 OPR 0 14 { writeln }
75    74 LIT 0 '5 mod 2 = ' { literal }
76    75 OPR 0 13 { write }
77    76 LIT 0 5 { literal }
78    77 LIT 0 2 { literal }
79    78 OPR 0 6 { binary mod }
80    79 OPR 0 14 { writeln }
```

```
81 80 LIT 0 'logic = ' { literal }
82 81 OPR 0 13 { write }
83 82 LOD 0 8 { load Variable }
84 83 OPR 0 14 { writeln }
85 84 LOD 0 4 { load Variable }
86 85 LIT 0 10 { literal }
87 86 OPR 0 7 { case compare }
88 87 JPC 0 92 { next case label }
89 88 JMP 0 89 { case matched }
90 89 LIT 0 1 { literal }
91 90 STO 0 6 { store Variable }
92 91 JMP 0 108 { end case }
93 92 LOD 0 4 { load Variable }
94 93 LIT 0 15 { literal }
95 94 OPR 0 7 { case compare }
96 95 JPC 0 100 { next case label }
97 96 JMP 0 97 { case matched }
98 97 LIT 0 2 { literal }
99 98 STO 0 6 { store Variable }
100 99 JMP 0 108 { end case }
101 100 LOD 0 4 { load Variable }
102 101 LIT 0 20 { literal }
103 102 OPR 0 7 { case compare }
104 103 JPC 0 108 { next case label }
105 104 JMP 0 105 { case matched }
106 105 LIT 0 3 { literal }
107 106 STO 0 6 { store Variable }
108 107 JMP 0 108 { end case }
109 108 LIT 0 'case branch = ' { literal }
110 109 OPR 0 13 { write }
111 110 LOD 0 6 { load Variable }
112 111 OPR 0 14 { writeln }
113 112 RET { return }
```

	Output Program	
	<pre> 1 sum = 15 2 5 / 2 = 2.5 3 5 div 2 = 2 4 5 mod 2 = 1 5 logic = true 6 case branch = 2 </pre>	
Hasil Uji Test Case 1:		Sesuai
2.	Input	

```
1  program Subprograms;
2
3  ✎ var
4    |   total: integer;
5    |   average, half: real;
6
7  procedure Add(delta: integer);
8  ✎ begin
9    |   total := total + delta
10 end;
11
12 function Inc(value: integer): integer;
13 ✎ begin
14   |   Inc := value + 1
15 end;
16
17 function Square(value: integer): integer;
18 ✎ begin
19   |   Square := value * value
20 end;
21
22 function AverageOf(left, right: integer): real;
23 ✎ begin
24   |   AverageOf := (left + right) / 2
25 end;
26
27 procedure ScaleAndStore(value: real);
28 ✎ begin
29   |   half := value / 2
30 end;
31
32 ✎ begin
33   |   total := 0;
34   |   Add(3);
35   |   Add(Square(Inc(2)));
```

```

36     average := AverageOf(total, 5);
37     ScaleAndStore(total);
38     writeln('total = ', total);
39     writeln('average = ', average);
40     writeln('half = ', half);
41     writeln('nested = ', Square(Inc(4)))
42 end.

```

Output Intermediate Code

```

1    0 JMP 0 41 { program entry }
2    1 JMP 0 2 { enter Add }
3    2 INT 0 4 { init frame Add }
4    3 LOD 1 3 { load Variable }
5    4 LOD 0 3 { load Variable }
6    5 OPR 0 2 { binary + }
7    6 STO 1 3 { store Variable }
8    7 RET { return Add }
9    8 JMP 0 9 { enter Inc }
10   9 INT 0 5 { init frame Inc }
11   10 LOD 0 3 { load Variable }
12   11 LIT 0 1 { literal }
13   12 OPR 0 2 { binary + }
14   13 STO 0 4 { store function result Inc }
15   14 LOD 0 4 { function result Inc }
16   15 RET { return Inc }
17   16 JMP 0 17 { enter Square }
18   17 INT 0 5 { init frame Square }
19   18 LOD 0 3 { load Variable }
20   19 LOD 0 3 { load Variable }

```

```
21    20 OPR 0 4 { binary * }
22    21 STO 0 4 { store function result Square }
23    22 LOD 0 4 { function result Square }
24    23 RET { return Square }
25    24 JMP 0 25 { enter AverageOf }
26    25 INT 0 6 { init frame AverageOf }
27    26 LOD 0 3 { load Variable }
28    27 LOD 0 4 { load Variable }
29    28 OPR 0 2 { binary + }
30    29 LIT 0 2 { literal }
31    30 OPR 0 5 { binary / }
32    31 STO 0 5 { store function result AverageOf }
33    32 LOD 0 5 { function result AverageOf }
34    33 RET { return AverageOf }
35    34 JMP 0 35 { enter ScaleAndStore }
36    35 INT 0 4 { init frame ScaleAndStore }
37    36 LOD 0 3 { load Variable }
38    37 LIT 0 2 { literal }
39    38 OPR 0 5 { binary / }
40    39 STO 1 5 { store Variable }
```

```
41 40 RET { return ScaleAndStore }
42 41 INT 0 6 { init program frame }
43 42 LIT 0 0 { literal }
44 43 STO 0 3 { store Variable }
45 44 LIT 0 3 { literal }
46 45 LIT 0 1 { argument count }
47 46 CAL 0 1 { call Add }
48 47 LIT 0 2 { literal }
49 48 LIT 0 1 { argument count }
50 49 CAL 0 8 { call Inc }
51 50 LIT 0 1 { argument count }
52 51 CAL 0 16 { call Square }
53 52 LIT 0 1 { argument count }
54 53 CAL 0 1 { call Add }
55 54 LOD 0 3 { load Variable }
56 55 LIT 0 5 { literal }
57 56 LIT 0 2 { argument count }
58 57 CAL 0 24 { call AverageOf }
59 58 STO 0 4 { store Variable }
60 59 LOD 0 3 { load Variable }
```

```

60 59 LOD 0 3 { load Variable }
61 60 OPR 0 16 { integer to real }
62 61 LIT 0 1 { argument count }
63 62 CAL 0 34 { call ScaleAndStore }
64 63 LIT 0 'total = ' { literal }
65 64 OPR 0 13 { write }
66 65 LOD 0 3 { load Variable }
67 66 OPR 0 14 { writeln }
68 67 LIT 0 'average = ' { literal }
69 68 OPR 0 13 { write }
70 69 LOD 0 4 { load Variable }
71 70 OPR 0 14 { writeln }
72 71 LIT 0 'half = ' { literal }
73 72 OPR 0 13 { write }
74 73 LOD 0 5 { load Variable }
75 74 OPR 0 14 { writeln }
76 75 LIT 0 'nested = ' { literal }
77 76 OPR 0 13 { write }
78 77 LIT 0 4 { literal }
79 78 LIT 0 1 { argument count }
80 79 CAL 0 8 { call Inc }
81 80 LIT 0 1 { argument count }
82 81 CAL 0 16 { call Square }
83 82 OPR 0 14 { writeln }
84 83 RET { return }

```

Output Program

```

1  total = 12
2  average = 8.5
3  half = 6
4  nested = 25

```

Hasil Uji Test Case 2:		Sesuai
3.	Input	

```
1  program Arrays;
2
3  type
4    Numbers == array[1 .. 5] of integer;
5    Matrix == array[1 .. 2] of array[1 .. 3] of integer;
6    CharScoreArray == array['a' .. 'c'] of integer;
7    BoolScoreArray == array[false .. true] of integer;
8
9  var
10   nums: Numbers;
11   grid: Matrix;
12   charScores: CharScoreArray;
13   flagScores: BoolScoreArray;
14   i, total: integer;
15   ch: char;
16   state: boolean;
17
18  begin
19   total := 0;
20   for i := 1 to 5 do
21     begin
22       nums[i] := i * i;
23       total := total + nums[i]
24     end;
25
26   grid[2, 3] := nums[5] + total;
27   ch := 'b';
28   charScores[ch] := grid[2, 3];
29   state := true;
30   flagScores[false] := 7;
31   flagScores[state] := charScores['b'] div 10;
32
33   writeln('array total = ', total);
34   writeln('matrix[2,3] = ', grid[2, 3]);
35   writeln('letters[b] = ', charScores[ch]);
36   writeln('flags[true] = ', flagScores[true]);
37   writeln('flags[false] = ', flagScores[false])
38  end.
```

Output Intermediate Code

```

1  0 JMP 0 1 { program entry }
2  1 INT 0 23 { init program frame }
3  2 LIT 0 0 { literal }
4  3 STO 0 20 { store Variable }
5  4 LIT 0 1 { literal }
6  5 STO 0 19 { store Identifier }
7  6 LOD 0 19 { load Identifier }
8  7 LIT 0 5 { literal }
9  8 OPR 0 12 { for condition }
10 9 JPC 0 29 { for false }
11 10 LOD 0 19 { load Variable }
12 11 LOD 0 19 { load Variable }
13 12 OPR 0 4 { binary * }
14 13 LDA 0 3 { address nums }
15 14 LOD 0 19 { load Variable }
16 15 IXA 1 5 1 { array index }
17 16 STI { store indirect ArrayAccess }
18 17 LOD 0 20 { load Variable }
19 18 LDA 0 3 { address nums }
20 19 LOD 0 19 { load Variable }
21 20 IXA 1 5 1 { array index }
22 21 LDI { load indirect ArrayAccess }
23 22 OPR 0 2 { binary + }
24 23 STO 0 20 { store Variable }
25 24 LOD 0 19 { load Identifier }
26 25 LIT 0 1 { for step }
27 26 OPR 0 2 { for update }
28 27 STO 0 19 { store Identifier }
29 28 JMP 0 6 { repeat for }
30 29 LOD 0 7 { load ArrayAccess }

```

```

30 29 LOD 0 7 { load ArrayAccess }
31 30 LOD 0 20 { load Variable }
32 31 OPR 0 2 { binary + }
33 32 STO 0 13 { store ArrayAccess }
34 33 LIT 0 'b' { literal }
35 34 STO 0 21 { store Variable }
36 35 LOD 0 13 { load ArrayAccess }
37 36 LDA 0 14 { address charScores }
38 37 LOD 0 21 { load Variable }
39 38 IXA 97 99 1 { array index }
40 39 STI { store indirect ArrayAccess }
41 40 LIT 0 true { literal }
42 41 STO 0 22 { store Variable }
43 42 LIT 0 7 { literal }
44 43 STO 0 17 { store ArrayAccess }
45 44 LOD 0 15 { load ArrayAccess }
46 45 LIT 0 10 { literal }
47 46 OPR 0 15 { binary div }
48 47 LDA 0 17 { address flagScores }
49 48 LOD 0 22 { load Variable }
50 49 IXA 0 1 1 { array index }
51 50 STI { store indirect ArrayAccess }
52 51 LIT 0 'array total = ' { literal }
53 52 OPR 0 13 { write }
54 53 LOD 0 20 { load Variable }
55 54 OPR 0 14 { writeln }
56 55 LIT 0 'matrix[2,3] = ' { literal }
57 56 OPR 0 13 { write }
58 57 LOD 0 13 { load ArrayAccess }
59 58 OPR 0 14 { writeln }
60 59 LIT 0 'letters[b] = ' { literal }

```

	<pre> 61 60 OPR 0 13 { write } 62 61 LDA 0 14 { address charScores } 63 62 LOD 0 21 { load Variable } 64 63 IXA 97 99 1 { array index } 65 64 LDI { load indirect ArrayAccess } 66 65 OPR 0 14 { writeln } 67 66 LIT 0 'flags[true] = ' { literal } 68 67 OPR 0 13 { write } 69 68 LOD 0 18 { load ArrayAccess } 70 69 OPR 0 14 { writeln } 71 70 LIT 0 'flags[false] = ' { literal } 72 71 OPR 0 13 { write } 73 72 LOD 0 17 { load ArrayAccess } 74 73 OPR 0 14 { writeln } 75 74 RET { return } </pre>		
	Output Program <pre> 1 array total = 55 2 matrix[2,3] = 80 3 letters[b] = 80 4 flags[true] = 8 5 flags[false] = 7 </pre>		
	<table> <tr> <td data-bbox="365 1413 1198 1497">Hasil Uji Test Case 3:</td> <td data-bbox="1198 1413 1430 1497">Sesuai</td> </tr> </table>	Hasil Uji Test Case 3:	Sesuai
Hasil Uji Test Case 3:	Sesuai		
4.	Input		

```
1  program Records;
2
3  type
4      Point == record
5          |   x, y: integer
6          | end;
7      Summary == record
8          |   total: integer;
9          |   average: real;
10         |   active: boolean
11         | end;
12
13  var
14      p: Point;
15      stats: Summary;
16      classification: integer;
17
18  begin
19      p.x := 6;
20      p.y := 8;
21      stats.total := p.x + p.y;
22      stats.average := stats.total / 2;
23      stats.active := stats.average >= 7;
24
25      if stats.active then
26          |   classification := 1
27          | else
28          |   classification := 0;
29
30      case classification of
31          |   0: writeln('record inactive');
32          |   1: writeln('record active')
33      end;
```

	<pre>34 35 writeln('point = ', p.x, ',', p.y); 36 writeln('total = ', stats.total); 37 writeln('average = ', stats.average) 38 end.</pre>
	Output Intermediate Code

```

1  0 JMP 0 1 { program entry }
2  1 INT 0 9 { init program frame }
3  2 LIT 0 6 { literal }
4  3 STO 0 3 { store FieldAccess }
5  4 LIT 0 8 { literal }
6  5 STO 0 4 { store FieldAccess }
7  6 LOD 0 3 { load FieldAccess }
8  7 LOD 0 4 { load FieldAccess }
9  8 OPR 0 2 { binary + }
10 9 STO 0 5 { store FieldAccess }
11 10 LOD 0 5 { load FieldAccess }
12 11 LIT 0 2 { literal }
13 12 OPR 0 5 { binary / }
14 13 STO 0 6 { store FieldAccess }
15 14 LOD 0 6 { load FieldAccess }
16 15 LIT 0 7 { literal }
17 16 OPR 0 10 { binary >= }
18 17 STO 0 7 { store FieldAccess }
19 18 LOD 0 7 { load FieldAccess }
20 19 JPC 0 23 { if false }
21 20 LIT 0 1 { literal }
22 21 STO 0 8 { store Variable }
23 22 JMP 0 25 { end if }
24 23 LIT 0 0 { literal }
25 24 STO 0 8 { store Variable }
26 25 LOD 0 8 { load Variable }
27 26 LIT 0 0 { literal }
28 27 OPR 0 7 { case compare }
29 28 JPC 0 33 { next case label }
30 29 JMP 0 30 { case matched }

```



```

31  30 LIT 0 'record inactive' { literal }
32  31 OPR 0 14 { writeln }
33  32 JMP 0 41 { end case }
34  33 LOD 0 8 { load Variable }
35  34 LIT 0 1 { literal }
36  35 OPR 0 7 { case compare }
37  36 JPC 0 41 { next case label }
38  37 JMP 0 38 { case matched }
39  38 LIT 0 'record active' { literal }
40  39 OPR 0 14 { writeln }
41  40 JMP 0 41 { end case }
42  41 LIT 0 'point = ' { literal }
43  42 OPR 0 13 { write }
44  43 LOD 0 3 { load FieldAccess }
45  44 OPR 0 13 { write }
46  45 LIT 0 ',' { literal }
47  46 OPR 0 13 { write }
48  47 LOD 0 4 { load FieldAccess }
49  48 OPR 0 14 { writeln }
50  49 LIT 0 'total = ' { literal }
51  50 OPR 0 13 { write }
52  51 LOD 0 5 { load FieldAccess }
53  52 OPR 0 14 { writeln }
54  53 LIT 0 'average = ' { literal }
55  54 OPR 0 13 { write }
56  55 LOD 0 6 { load FieldAccess }
57  56 OPR 0 14 { writeln }
58  57 RET { return }

```

Output Program

	<pre>1 record active 2 point = 6,8 3 total = 14 4 average = 7</pre>	
Hasil Uji Test Case 4:		Sesuai
5.	Input	

```
1  program SquareRoot;
2
3  function AbsValue(x: real): real;
4  begin
5      if x >= 0 then
6          AbsValue := x
7      else
8          AbsValue := -x
9  end;
10
11 function SqrtApprox(x: real): real;
12 var
13     left, right: real;
14 begin
15     left := 0;
16     right := x;
17     SqrtApprox := 1.0;
18     repeat
19         if SqrtApprox * SqrtApprox > x then
20             right := SqrtApprox
21         else
22             left := SqrtApprox;
23             SqrtApprox := (left + right) / 2
24     until AbsValue(SqrtApprox * SqrtApprox - x) < 0.0001
25 end;
26
27 begin
28     writeln('sqrt(9) = ', SqrtApprox(9));
29     writeln('sqrt(2) = ', SqrtApprox(2));
30     writeln('sqrt(4/9) = ', SqrtApprox(4 / 9));
31     writeln('sqrt(0.01) = ', SqrtApprox(0.01))
32 end.
```

Output Intermediate Code

```
1  0 JMP 0 54 { program entry }
2  1 JMP 0 2 { enter AbsValue }
3  2 INT 0 5 { init frame AbsValue }
4  3 LOD 0 3 { load Variable }
5  4 LIT 0 0 { literal }
6  5 OPR 0 10 { binary >= }
7  6 JPC 0 10 { if false }
8  7 LOD 0 3 { load Variable }
9  8 STO 0 4 { store function result AbsValue }
10 9 JMP 0 14 { end if }
11 10 LOD 0 3 { load Variable }
12 11 OPR 0 1 { unary minus }
13 12 OPR 0 16 { integer to real }
14 13 STO 0 4 { store function result AbsValue }
15 14 LOD 0 4 { function result AbsValue }
16 15 RET { return AbsValue }
17 16 JMP 0 17 { enter SqrtApprox }
18 17 INT 0 7 { init frame SqrtApprox }
19 18 LIT 0 0 { literal }
20 19 OPR 0 16 { integer to real }
21 20 STO 0 4 { store Variable }
22 21 LOD 0 3 { load Variable }
23 22 STO 0 5 { store Variable }
24 23 LIT 0 1.0 { literal }
25 24 STO 0 6 { store function result SqrtApprox }
26 25 LOD 0 6 { load function result SqrtApprox }
27 26 LOD 0 6 { load function result SqrtApprox }
28 27 OPR 0 4 { binary * }
29 28 LOD 0 3 { load Variable }
30 29 OPR 0 11 { binary > }
31 30 JPC 0 34 { if false }
32 31 LOD 0 6 { load function result SqrtApprox }
33 32 STO 0 5 { store Variable }
34 33 JMP 0 36 { end if }
35 34 LOD 0 6 { load function result SqrtApprox }
```

```

36 35 STO 0 4 { store Variable }
37 36 LOD 0 4 { load Variable }
38 37 LOD 0 5 { load Variable }
39 38 OPR 0 2 { binary + }
40 39 LIT 0 2 { literal }
41 40 OPR 0 5 { binary / }
42 41 STO 0 6 { store function result SqrtApprox }
43 42 LOD 0 6 { load function result SqrtApprox }
44 43 LOD 0 6 { load function result SqrtApprox }
45 44 OPR 0 4 { binary * }
46 45 LOD 0 3 { load Variable }
47 46 OPR 0 3 { binary - }
48 47 LIT 0 1 { argument count }
49 48 CAL 1 1 { call AbsValue }
50 49 LIT 0 0.0001 { literal }
51 50 OPR 0 9 { binary < }
52 51 JPC 0 25 { repeat until false }
53 52 LOD 0 6 { function result SqrtApprox }
54 53 RET { return SqrtApprox }
55 54 INT 0 3 { init program frame }
56 55 LIT 0 'sqrt(9) = ' { literal }
57 56 OPR 0 13 { write }
58 57 LIT 0 9 { literal }
59 58 OPR 0 16 { integer to real }
60 59 LIT 0 1 { argument count }
61 60 CAL 0 16 { call SqrtApprox }
62 61 OPR 0 14 { writeln }
63 62 LIT 0 'sqrt(2) = ' { literal }
64 63 OPR 0 13 { write }
65 64 LIT 0 2 { literal }
66 65 OPR 0 16 { integer to real }
67 66 LIT 0 1 { argument count }
68 67 CAL 0 16 { call SqrtApprox }
69 68 OPR 0 14 { writeln }
70 69 LIT 0 'sqrt(4/9) = ' { literal }

```

	<pre> 71 70 OPR 0 13 { write } 72 71 LIT 0 4 { literal } 73 72 LIT 0 9 { literal } 74 73 OPR 0 5 { binary / } 75 74 LIT 0 1 { argument count } 76 75 CAL 0 16 { call SqrtApprox } 77 76 OPR 0 14 { writeln } 78 77 LIT 0 'sqrt(0.01) = ' { literal } 79 78 OPR 0 13 { write } 80 79 LIT 0 0.01 { literal } 81 80 LIT 0 1 { argument count } 82 81 CAL 0 16 { call SqrtApprox } 83 82 OPR 0 14 { writeln } 84 83 RET { return } </pre>
	Output Program
	<pre> 1 sqrt(9) = 3 2 sqrt(2) = 1.41418 3 sqrt(4/9) = 0.666626 4 sqrt(0.01) = 0.0996094 </pre>
Hasil Uji Test Case 5:	
Belum Sesuai	

Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan pada Milestone 4 tugas besar Arion Compiler, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Compiler/Interpreter Arion pada Milestone 4 sudah berhasil menjalankan alur penuh dari *source code* hingga eksekusi melalui interpreter: Tokenizer → Parser → AST → Decorated AST → Intermediate Code → Interpreter.
2. Intermediate code berbasis *stack machine* sudah dapat merepresentasikan dan menjalankan fitur utama bahasa Arion, seperti assignment, ekspresi, percabangan, perulangan, procedure/function, array, record, dan write/writeln.
3. Interpreter sudah mendukung eksekusi runtime dengan penanganan operasi penting seperti real division /, integer division div, pemanggilan function dengan return melalui assignment ke nama function, serta akses array dan record.

B. Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan tahap berikutnya:

1. Perbanyak test case negatif untuk menguji error handling, seperti division by zero, array index out of bounds, argumen function/procedure yang salah, dan akses field record yang tidak valid.
2. Dokumentasikan instruksi tambahan di luar spesifikasi dasar, seperti RDIV, IDIV, dan I2R, termasuk alasan penambahannya dalam rancangan intermediate code.
3. Tambahkan pengujian otomatis dan dokumentasi runtime stack/call frame agar pengembangan interpreter berikutnya lebih mudah diverifikasi dan dipahami.

Lampiran

Link repository Github: <https://github.com/NafisKreatif/RGX-Tubes-IF2224-2026>

Link Diagram Flowchart:

<https://drive.google.com/file/d/112w4aT5AhRnhmSPkPaetoq4s52rJPNcC/view?usp=sharing>

Tabel Pembagian Tugas

No.	Nama	NIM	Pembagian Tugas	Kontribusi (%)
1	Wafiq Hibban Robbany	13524016	IntermediateCode, IntermediateCodeGenerato r	26
2	Muhammad Nafis Habibi	13524018	Intermediate Code, Testing	22
3	An-Dafa Anza Avansyah	13524038	Interpreter, Testing	26
4	Wildan Abdurrahman Ghazali	13524054	IntermediateCode, IntermediateCodeGenerato r	26

Referensi

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.